IA propia · Qué falta para integrarla

Oct 12, 2026 · @German

El backend ya está listo para el modo vs. IA propia: solo falta el servicio Python de Desarrollo Simulación (Paula, Lázaro, Luciano) en el puerto 8000. Cuando la IA no conoce una posición, el backend ya usa Stockfish a nivel 0 como respaldo, y eso funciona hoy. Backend no desarrolló ni modificó nada de la IA propia: solo la parte que le toca, que es llamar al servicio con el contrato que fijó Diseño Técnico. Este documento junta lo que Simulación ya anotó como pendiente y lo que el backend espera recibir; el diseño de la IA es de ellos.

Contrato con el backend

El backend llama a `POST {AI_SERVICE_URL}/predict-move` (por defecto `http://127.0.0.1:8000`) y espera la respuesta en menos de 3000 ms (`AI_TIMEOUT_MS`).

Pedido:

```
{ "fen": "rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq - 0 1",
"difficulty": "MEDIO", "elo": 1300, "gameId": "uuid" }
```

Respuesta:

```
{ "move": "e7e5", "source": "history" }
```

Campo	Valores	Nota
move	Jugada en UCI: e7e5, o e7e8q si corona	Tiene que ser legal en ese FEN; si no, el backend responde 502 ENGINE_BAD_MOVE
source	history, openings o none	none = no conoce la posición: el backend juega con Stockfish nivel 0
difficulty	FACIL, MEDIO, DIFICIL	Políticas de RN-02: Fácil variantes débiles, Medio ruleta ponderada, Difícil la de mayor tasa de victorias
fen	FEN completo de 6 campos	Para buscar, usar los primeros 4 (posición, turno, enroques, al paso)

Si el servicio no está levantado, el backend responde `503 ENGINE_UNAVAILABLE`; si tarda más de 3 s, `504 ENGINE_TIMEOUT`.

Lo que falta, en orden (tareas de Simulación, según su pestaña)

- [ ] Servicio Python mínimo (FastAPI) en el puerto 8000, con `GET /health` y `POST /predict-move` devolviendo una jugada legal al azar y `source: "history"`. Con esto ya se puede jugar contra la "IA" de punta a punta.
- [ ] Acordar con GC y Modelado de Datos qué base se usa y crearla con `pnpm migrate` (tablas games, moves y la vista v_training_moves).
- [ ] Script de carga del dataset de Lichess: cada partida en games con `source = 'SEED'` y `status = 'FINISHED'`, cada jugada en moves (san, uci, fen_before, fen_after). Ver la sección siguiente.
- [ ] Que /predict-move busque en la base las jugadas de esa posición (`fen_before_key`, ya indexado) y aplique las políticas de RN-02 por dificultad. Para estadísticas usar la vista `v_training_moves`, que solo trae partidas finalizadas.
- [ ] Responder `source: "none"` cuando no haya datos, para que el backend use Stockfish de respaldo.
- [ ] `POST /model/train` (reentrenar con las partidas nuevas de usuarios) y registrar cada versión en ai_model_versions.
- [ ] Pruebas: que Difícil elija siempre la de mayor tasa de victorias, que Medio reparta según la ruleta y que el respaldo se active sin datos.
- [ ] Pendiente con el profesor: si la dificultad va por ELO o por nivel. La tabla ya tiene los dos datos.

Cómo procesar los PGN de Lichess

Es la duda que Simulación anotó en su pestaña. Esto es solo una sugerencia de Backend; la decisión es de ellos. La clave es no cargar nunca el archivo entero:

1. Leer de a una partida con `python-chess`: `chess.pgn.read_game(archivo)` devuelve la siguiente partida del archivo abierto.
2. Si el archivo viene comprimido (`.pgn.zst`), descomprimirlo mientras se lee con el paquete `zstandard` (`stream_reader`).
3. Filtrar por los encabezados `WhiteElo` y `BlackElo` antes de recorrer las jugadas: sirve para separar las dificultades y para no cargar los 10 millones.
4. Recorrer `game.mainline_moves()` sobre un `chess.Board()`, guardando por jugada el FEN antes, la jugada en SAN y UCI, y el FEN después.

5. Insertar por lotes (por ejemplo cada 1.000 partidas) en una transacción, con `psycopg` y `executemany`.

Conviene empezar con unas 100.000 partidas filtradas, medir cuántas posiciones cubren y recién después cargar más.